

Exposure Fusion

Studio e implementazione in Python

Analisi del paper, mappatura sul codice e risultati sperimentali



Documento tecnico sull'implementazione dell'algoritmo di Exposure Fusion di Mertens, Kautz e Van Reeth, con formule, corrispondenza al codice e risultati su dataset di esposizioni multiple.

Progetto realizzato per il corso di Computer Graphics della Laurea Magistrale in Informatica, tenuto dal Prof. Fabio Pellacini presso l'Università degli Studi di Modena e Reggio Emilia (Unimore).

Angelo Longano

Laurea Magistrale in Informatica - Unimore

7 giugno 2026

Indice

1	Abstract	2
2	Introduzione	2
3	Metodo	3
4	Struttura dell'implementazione	4
5	Calcolo delle mappe di peso	5
5.1	Contrasto	5
5.2	Saturazione cromatica	6
5.3	Well-exposedness	6
6	Normalizzazione dei pesi	7
7	Fusione multirisoluzione	7
8	Scelta della profondità della piramide	8
9	Dataset e risultati	9
9.1	Venice Boat	10
9.2	Venice Carnival	10
9.3	Living Room Window	11
10	Controlli numerici	12
10.1	Normalizzazione	13
10.2	Ricostruzione della piramide Laplaciana	13
10.3	Coerenza delle shape	13
11	Differenze implementative e limiti	14
11.1	Elementi non implementati rispetto al paper	14
12	Conclusioni	15
13	Riferimenti	15

1 Abstract

Questo documento presenta lo studio e l'implementazione in Python dell'algoritmo **Exposure Fusion** proposto da Mertens, Kautz e Van Reeth. Il metodo combina una sequenza di immagini LDR acquisite con esposizioni differenti in una singola immagine LDR finale, senza ricostruire una radiance map HDR e senza richiedere stima della Camera Response Function o tone mapping.

L'implementazione realizzata riproduce il nucleo algoritmico del paper: per ogni immagine vengono calcolate misure locali di qualità, queste misure vengono combinate in mappe di peso normalizzate e la fusione finale viene eseguita in multirisoluzione tramite piramidi Laplaciane delle immagini e piramidi Gaussiane dei pesi. Il progetto non mira a essere una replica bit-exact del codice originale degli autori, ma a rendere esplicita la corrispondenza tra formule, codice Python e risultati visivi.

2 Introduzione

Una singola immagine LDR (*Low Dynamic Range*) non è sempre sufficiente per rappresentare scene con alto range dinamico. In presenza di regioni molto scure e molto luminose, una sola esposizione tende a perdere informazione: le ombre possono risultare sottoesposte, mentre le alte luci possono essere clippate.

La fotografia HDR classica affronta il problema acquisendo una sequenza di esposizioni e stimando una rappresentazione radiometrica della scena. Questa rappresentazione viene poi trasformata in una immagine visualizzabile tramite tone mapping:

`bracketed exposures -> HDR reconstruction -> tone mapping -> display image`

Exposure Fusion propone una pipeline più diretta. Invece di stimare una radiance map o una rappresentazione radiometrica HDR della scena, assegna pesi locali ai pixel delle immagini LDR e li combina in una immagine finale:

`bracketed exposures -> weight maps -> multiresolution blending -> display image`

Il vantaggio principale è la semplificazione: il metodo non richiede tempi di esposizione, calibrazione radiometrica o stima della curva di risposta della camera. Il risultato finale è una immagine LDR visivamente plausibile, non una radiance map HDR.

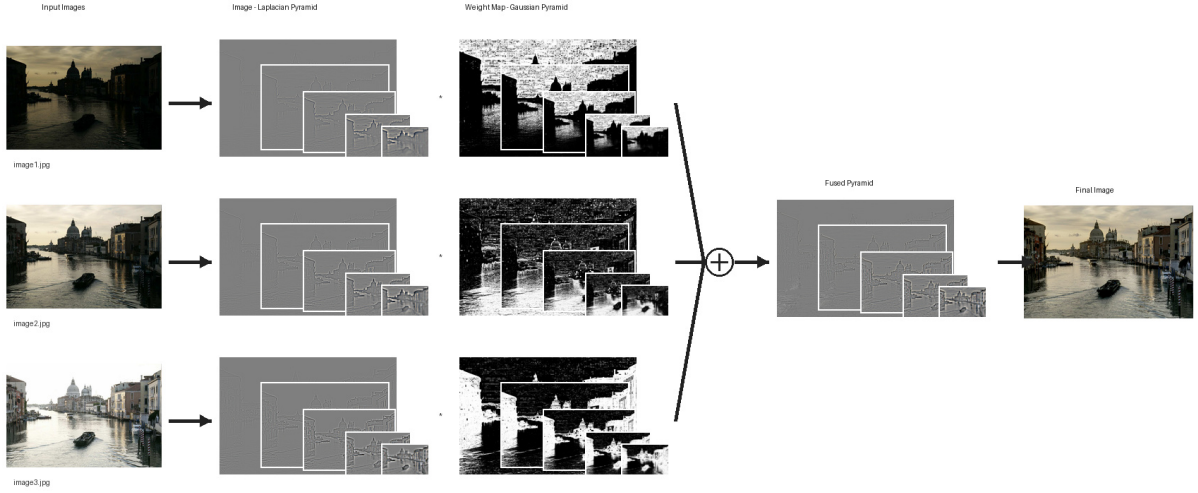


Figura 1: Pipeline completa dell'implementazione sul dataset Venice Boat.

3 Metodo

L'idea di base del paper è che, in una sequenza di esposizioni, ogni immagine contiene regioni localmente più utili di altre. Una esposizione scura può preservare dettagli nelle alte luci; una esposizione chiara può rendere visibili dettagli nelle ombre. L'algoritmo assegna quindi a ogni pixel un peso che misura quanto quel pixel sia adatto a contribuire al risultato finale.

Per ogni pixel (i, j) dell'immagine k , vengono calcolate tre misure:

- **contrasto**, che favorisce bordi, texture e variazioni locali;
- **saturazione cromatica**, che favorisce colori non sbiaditi;
- **well-exposedness**, che favorisce valori di intensità intermedi.

Le tre misure vengono combinate tramite prodotto:

$$W_{i,j,k} = C_{i,j,k}^{\omega_C} S_{i,j,k}^{\omega_S} E_{i,j,k}^{\omega_E}$$

dove C , S ed E indicano contrasto, saturazione cromatica e well-exposedness, mentre gli esponenti ω_C , ω_S e ω_E controllano l'importanza relativa delle componenti. Nell'implementazione di default tutti gli esponenti valgono 1.0.

I pesi grezzi vengono poi normalizzati lungo la sequenza:

$$\hat{W}_{i,j,k} = \frac{W_{i,j,k}}{\sum_n W_{i,j,n}}$$

in modo che, per ogni pixel, la somma dei contributi delle diverse esposizioni sia pari a uno:

$$\sum_k \hat{W}_{i,j,k} = 1.$$

Una fusione pixel-wise diretta può essere scritta come:

$$R_{i,j} = \sum_k \hat{W}_{i,j,k} I_{i,j,k}.$$

Questa forma è semplice, ma può produrre transizioni innaturali quando le mappe di peso cambiano bruscamente o quando le esposizioni hanno luminosità globali molto diverse. In questa sola formulazione diretta, la normalizzazione dei pesi rende la combinazione pixel-wise convessa rispetto ai valori RGB di input. Il risultato multirisoluzione usato nel progetto non è però semplicemente questa media locale: fonde coefficienti Laplaciani a scale diverse e poi collassa la piramide risultante. Per ridurre gli artefatti, infatti, il paper combina le immagini in multirisoluzione: le immagini vengono decomposte in piramidi Laplaciane, i pesi in piramidi Gaussiane, e la fusione avviene livello per livello.

4 Struttura dell'implementazione

Il progetto separa il codice in moduli che corrispondono alle fasi dell'algoritmo. La tabella seguente riassume la relazione tra concetti del paper e implementazione.

Concetto	Ruolo nell'algoritmo	Implementazione
Misure di qualità	Contrasto, saturazione cromatica, well-exposedness	<code>exposure_fusion_core/weights.py</code> , <code>quality_measures</code>
Formula dei pesi	Prodotto $C^{\omega_C} S^{\omega_S} E^{\omega_E}$	<code>weight_map</code>
Normalizzazione	Somma pixel-wise dei pesi pari a 1	<code>normalize_weights</code>
Piramidi	Decomposizione Gaussiana e Laplaciana	<code>exposure_fusion_core/pyramids.py</code>
Fusione	Blending livello per livello	<code>exposure_fusion_core/fusion.py</code> , <code>fuse_exposures</code>

L'interfaccia principale è `fuse_exposures`. La funzione riceve una lista di immagini RGB con la stessa shape, caricate come array NumPy in virgola mobile con valori in $[0, 1]$, e restituisce una struttura `ExposureFusionResult`. I JPEG vengono quindi trattati come valori RGB display-referred già pronti per l'elaborazione: il codice non linearizza sRGB, non applica una gamma inversa e non gestisce profili colore. Questa scelta è coerente con l'obiettivo didattico del progetto, ma va

distinta da una pipeline radiometricamente calibrata. La struttura contiene l'immagine finale, ma anche pesi grezzi, pesi normalizzati, piramidi delle immagini, piramidi dei pesi e piramide Laplaciana fusa. La scelta rende la pipeline ispezionabile e utile per uno studio didattico dell'algoritmo.

L'entry point `main.py` permette di eseguire la pipeline da riga di comando e di salvare artefatti intermedi, come input LDR, mappe di peso, confronto con un riferimento e diagramma del processo. Un esempio di uso per rigenerare anche le anteprime aggregate mostrate in questo documento è:

```
uv run python main.py \
  --inputs images/venice_boat/image1.jpg \
    images/venice_boat/image2.jpg \
    images/venice_boat/image3.jpg \
  --output-dir images/venice_boat/out \
  --reference images/venice_boat/result.jpg \
  --pyramid-max-layer 8 \
  --save-all \
  --save-process \
  --preview
```

I dataset e gli output previsti sono dichiarati in `paper_sets.toml`. I casi discussi visivamente nel documento sono `venice_boat`, `venice_carnival` e `living_room_window`.

5 Calcolo delle mappe di peso

La prima fase della pipeline calcola una mappa di peso per ogni immagine. Le immagini sono convertite in RGB e normalizzate nell'intervallo $[0, 1]$ tramite `load_rgb_float`.

5.1 Contrasto

Il contrasto viene stimato applicando un filtro Laplaciano all'immagine in scala di grigi:

```
LAPLACIAN_KERNEL = np.array(
    [
        [0.0, 1.0, 0.0],
        [1.0, -4.0, 1.0],
        [0.0, 1.0, 0.0],
    ]
)

gray = rgb2gray(image)
contrast = np.abs(convolve(gray, LAPLACIAN_KERNEL, mode="reflect"))
```

La risposta assoluta del Laplaciano evidenzia bordi e variazioni locali. La gestione dei bordi usa `mode="reflect"`, quindi i valori vicino ai margini dipendono da questa specifica convenzione di convoluzione. Il punto importante è che la misura favorisce dettaglio locale; non va interpretata

come una misura radiometrica della scena. Il kernel usato è la variante a 4-neighbor, non quella a 8-neighbor.

5.2 Saturazione cromatica

La saturazione cromatica viene stimata come deviazione standard dei canali RGB:

```
saturation = np.std(image, axis=2)
```

Questa è una proxy semplice e locale. Un pixel con canali simili tende ad avere bassa saturazione; un pixel con canali più differenziati tende ad avere una risposta maggiore. Non si tratta di un modello percettivo completo della saturazione colore. Il termine non va confuso con il clipping delle alte luci: nel contesto delle mappe di peso, “saturazione” indica solo la differenziazione cromatica tra i canali RGB.

5.3 Well-exposedness

La well-exposedness favorisce intensità intermedie e penalizza valori vicini a 0 o 1. L’implementazione usa una curva Gaussiana centrata in 0.5:

$$E(x) = \exp\left(-\frac{(x - 0.5)^2}{2\sigma^2}\right).$$

La curva viene applicata ai canali RGB e i risultati vengono moltiplicati:

```
well_exposedness = np.prod(
    exposedness_curve(image, sigma=config.sigma_exposedness),
    axis=2,
)
```

Con la configurazione di default, `sigma_exposedness` vale 0.2. Il peso finale è il prodotto delle tre misure:

```
weight = (
    contrast**config.omega_contrast
    * saturation**config.omega_saturation
    * well_exposedness**config.omega_exposedness
)
```

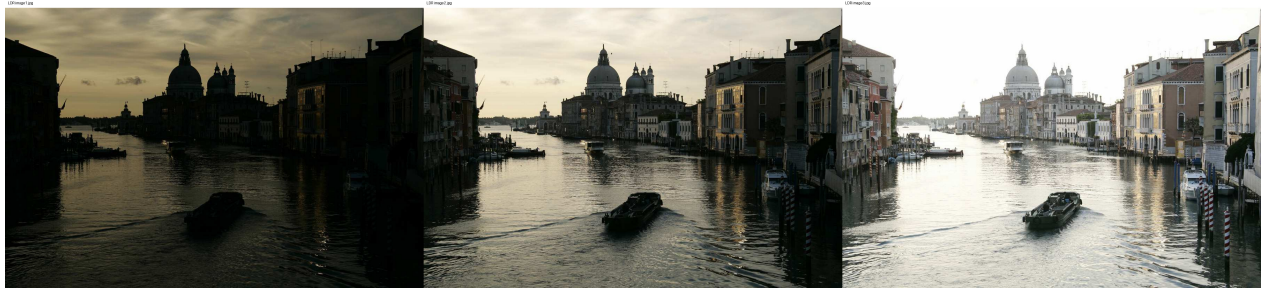


Figura 2: Esposizioni di input per il dataset Venice Boat.



Figura 3: Mappe di peso normalizzate per il dataset Venice Boat.

6 Normalizzazione dei pesi

Dopo il calcolo dei pesi grezzi, le mappe vengono normalizzate lungo la dimensione delle immagini. Se `weights` ha forma:

```
(num_images, height, width)
```

la normalizzazione avviene lungo `axis=0`:

```
weight_sum = weights.sum(axis=0, keepdims=True)
uniform = np.full_like(weights, 1.0 / weights.shape[0])
normalized = np.divide(weights, weight_sum, out=uniform, where=weight_sum > eps)
```

Il fallback uniforme viene usato quando la somma locale dei pesi è nulla o numericamente degenerata. In quel caso, invece di dividere per zero, ogni immagine riceve lo stesso contributo. Questo controllo evita che la fusione sia indefinita nei pixel in cui tutte le misure producono peso nullo.

7 Fusione multirisoluzione

La fusione finale segue la struttura del paper: non combina direttamente i pixel, ma combina coefficienti a scale diverse. Per ogni immagine viene costruita una piramide Laplaciana; per ogni mappa di peso normalizzata viene costruita una piramide Gaussiana.

Per ogni livello l :

$$L_l(R) = \sum_k G_l(\hat{W}_k) \cdot L_l(I_k)$$

dove $L_l(I_k)$ è il livello l della piramide Laplaciana dell'immagine k , $G_l(\hat{W}_k)$ è il livello l della piramide Gaussiana del peso normalizzato, e $L_l(R)$ è il livello fuso. La piramide risultante viene poi collassata per ottenere l'immagine finale.

Le piramidi sono costruite con funzioni di `scikit-image`: `pyramid_gaussian` e `pyramid_expand`. Poiché downsampling e upsampling possono introdurre differenze di shape sui bordi, `crop_like` riallinea l'immagine espansa alla shape del livello di riferimento.

Dopo aver costruito le piramidi Gaussiani dei pesi, `fuse_exposures` rinormalizza i pesi a ogni livello. Questa è una scelta implementativa aggiunta per stabilità numerica: nel paper la normalizzazione è definita prima della costruzione delle piramidi, mentre qui viene ripetuta dopo il downsampling per ridurre piccoli drift numerici dovuti al filtraggio e al ridimensionamento.

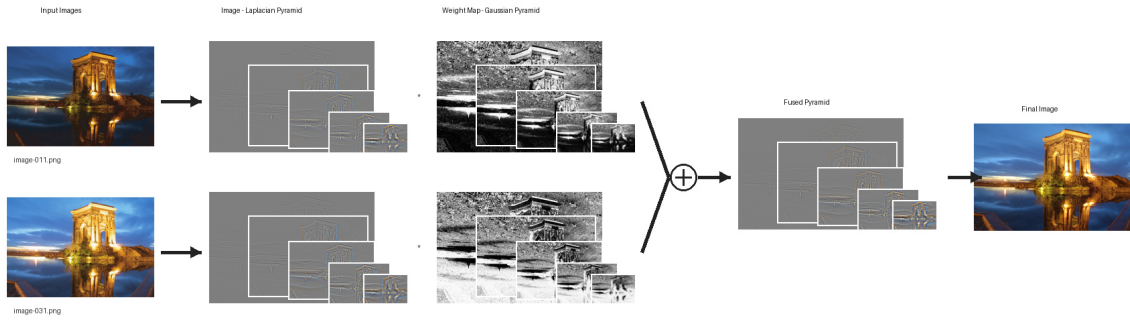


Figura 4: Schema di piramidi Gaussiani, piramidi Laplaciane e fusione multiscala.

8 Scelta della profondità della piramide

La profondità della piramide influenza il risultato finale: una piramide troppo corta combina soprattutto dettagli locali, mentre una piramide completa scende fino a scale molto grossolane e può introdurre differenze globali di luminosità rispetto ai riferimenti disponibili. Per questo motivo è stata eseguita una analisi di sensibilità sul parametro `max_layer`.

Per ogni dataset con riferimento, lo script `scripts/sweep_pyramid_layers.py` prova diversi valori di `max_layer`, genera la fusione e calcola MAE e MSE rispetto alla reference scelta. MAE e MSE sono quindi distanze numeriche dal riferimento disponibile, non misure di ground truth radiometrica, qualità percettiva assoluta o correttezza HDR. Questa analisi è una procedura sperimentale del

progetto e non fa parte del metodo originale di Mertens et al. Il valore `-1` indica la piramide completa fino alla scala minima permessa da `scikit-image`; con `max_layer=N`, invece, `scikit-image` può produrre fino a `N+1` livelli, includendo il livello originale. Questa procedura non identifica un parametro universalmente ottimo: misura quale profondità approssima meglio il riferimento per il singolo dataset.

Dataset	max_layer completo	MAE completo	MSE completo	max_layer scelto	MAE scelto	MSE scelto
venice_boat	-1	0.067872	0.007582	8	0.025263	0.001097
venice_carnival	-1	0.092841	0.011839	8	0.031380	0.001622
living_room_window	-1	0.115918	0.020146	6	0.049332	0.003852

Gli asset finali del paper sono stati rigenerati usando questi valori: `max_layer=8` per `venice_boat`, `max_layer=8` per `venice_carnival` e `max_layer=6` per `living_room_window`. L'esempio personale `iphone_example_2`, non incluso nelle figure principali, ha invece ottenuto la metrica migliore con la piramide completa (`max_layer=-1`), confermando che il parametro dipende dal contenuto dell'immagine e dalla reference usata. Gli stessi valori sono riportati anche in `paper_sets.toml`, insieme agli input e alle reference dei dataset.

Le metriche della tabella precedente sono calcolate su `fused_image`, cioè sul risultato già clippato in $[0, 1]$ da `fuse_exposures`. Questo è coerente con le immagini salvate e confrontate visivamente, ma può nascondere overshoot o undershoot presenti nella fusione grezza `fused_raw`. Per questo sono riportati anche il range del risultato grezzo, la percentuale di pixel che subiscono clipping e la percentuale di pixel in cui la normalizzazione dei pesi iniziali usa il fallback uniforme.

Dataset	Livelli	raw_min	raw_max	Pixel clippati	Fallback uniforme	Reference ridimen- sionata
venice_boat	9	-0.296183	1.224675	5.493507%	3.316354%	no
venice_carnival	9	-0.316631	1.510505	1.945486%	2.462604%	no
living_room_window	7	-0.220163	1.273979	1.102675%	5.245679%	sì

Per `living_room_window` la reference ha dimensioni diverse dagli input e viene ridimensionata alla shape della sequenza prima del confronto; i valori numerici vanno quindi letti come indicativi.

9 Dataset e risultati

Gli esperimenti documentati nel paper usano tre dataset di fusione (`venice_boat`, `venice_carnival` e `living_room_window`) e un dataset di supporto per spiegare la costruzione delle piramidi

(`pyramid_blending`). La configurazione in `paper_sets.toml` contiene anche esempi personali opzionali non discussi nella relazione. Gli output sono salvati nelle rispettive cartelle `out/`, in modo da mantenere separati input originali e artefatti generati. Le figure mostrate usano la profondità di piramide scelta tramite l'analisi quantitativa precedente.

9.1 Venice Boat

`venice_boat` contiene tre esposizioni di una scena esterna. Il dataset è utile per mostrare come esposizioni differenti contribuiscano a regioni diverse dell'immagine finale. La figura seguente confronta il risultato della fusione con il riferimento disponibile e con una mappa di differenza assoluta.



Figura 5: Confronto finale per il dataset Venice Boat.

La valutazione resta qualitativa: il riferimento non dimostra una correttezza radiometrica assoluta, ma permette di ispezionare visivamente luminosità, contrasto e differenze locali. Nelle figure di confronto, la mappa di differenza mostra la media del valore assoluto sui tre canali RGB e viene visualizzata con `difference_vmax=0.25`: differenze pari o superiori a 0.25 nella scala $[0, 1]$ vengono quindi clippate nella visualizzazione.

9.2 Venice Carnival

`venice_carnival` contiene tre esposizioni di una scena con un soggetto in maschera in Piazza San Marco. Rispetto a `venice_boat`, il caso presenta colori cromaticamente più saturi e regioni con texture più evidenti; per questo è utile per ispezionare il comportamento combinato di saturazione cromatica, contrasto e well-exposedness.



Figura 6: Esposizioni di input per il dataset Venice Carnival.

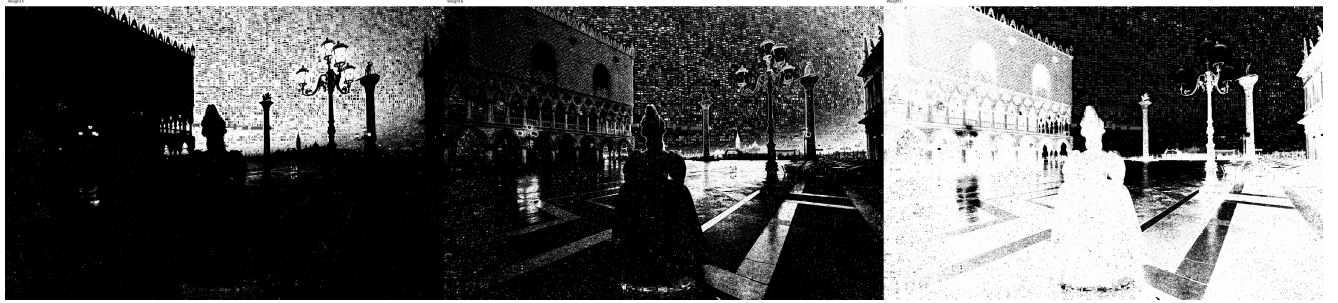


Figura 7: Mappe di peso normalizzate per il dataset Venice Carnival.



Figura 8: Confronto finale per il dataset Venice Carnival.

Nel confronto finale, la fusione conserva il soggetto principale e combina le diverse esposizioni per ridurre sia le zone troppo scure sia le regioni troppo chiare. La scena è anche un buon caso di controllo visivo per eventuali artefatti: colori intensi, bordi della maschera e dettagli dello sfondo rendono più evidente se le mappe di peso o il blending multirisoluzione introducono transizioni innaturali.

9.3 Living Room Window

`living_room_window` contiene quattro immagini di una scena interna con una finestra luminosa. Il caso è rappresentativo perché contiene una forte differenza tra dettagli nelle zone interne scure e regione esterna molto luminosa.



Figura 9: Esposizioni di input per il dataset Living Room Window.

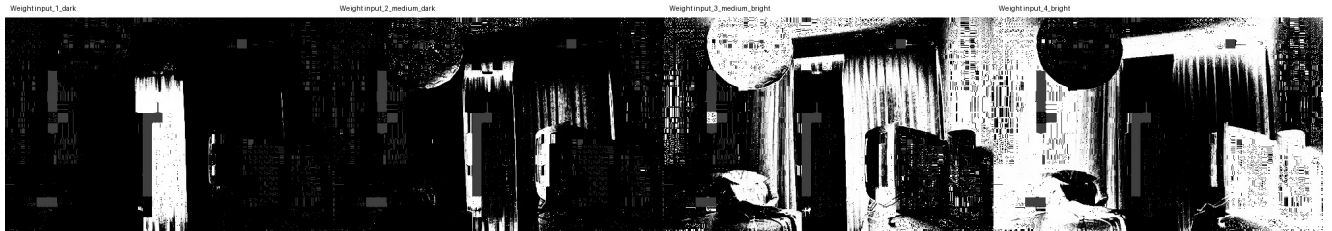


Figura 10: Mappe di peso normalizzate per il dataset Living Room Window.

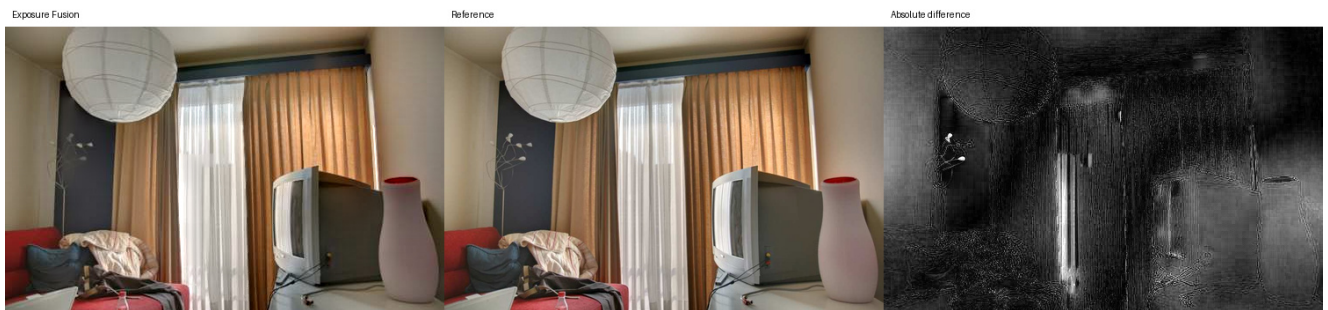


Figura 11: Confronto finale per il dataset Living Room Window.

Le mappe di peso mostrano qualitativamente il ruolo della well-exposedness: le esposizioni scure tendono a contribuire nelle alte luci, mentre quelle più chiare contribuiscono nelle aree interne. La fusione multirisoluzione combina questi contributi cercando transizioni più regolari rispetto a una fusione pixel-wise semplice.

10 Controlli numerici

La qualità finale di Exposure Fusion è in parte percettiva, quindi non può essere riassunta da una sola metrica numerica. I controlli seguenti non dimostrano da soli la correttezza visiva o percettiva del risultato: verificano invece la coerenza numerica e implementativa, cioè che i pesi siano normalizzati, che le piramidi siano ricostruibili e che le shape dei livelli siano compatibili.

10.1 Normalizzazione

Per ogni pixel, la somma dei pesi normalizzati lungo la sequenza deve essere circa uguale a 1:

$$\max_{i,j} \left| \sum_k \hat{W}_{i,j,k} - 1 \right| \approx 0.$$

Il controllo è stato calcolato sui dataset di fusione disponibili. La colonna dei pesi piramidali riporta il massimo errore osservato dopo la rinormalizzazione dei livelli Gaussiani.

Dataset	Livelli	Errore pesi iniziali	Errore pesi piramidali
venice_boat	9	$3.331 \cdot 10^{-16}$	$2.220 \cdot 10^{-16}$
living_room_window	7	$3.331 \cdot 10^{-16}$	$3.331 \cdot 10^{-16}$
venice_carnival	9	$3.331 \cdot 10^{-16}$	$2.220 \cdot 10^{-16}$

Il fallback uniforme nella normalizzazione iniziale dei pesi non è un errore: si attiva nei pixel in cui la somma dei pesi grezzi è nulla o inferiore alla soglia numerica **eps=1e-12**. Nei dataset principali la sua incidenza è quella riportata nella tabella delle statistiche sul risultato scelto.

10.2 Ricostruzione della piramide Laplaciana

Un secondo controllo verifica che la costruzione e il collasso della piramide Laplaciana preservino l'immagine originale entro la tolleranza numerica:

$$\text{collapse}(\text{laplacian_pyramid}(I)) \approx I.$$

Sulla prima immagine di ciascun dataset, l'errore massimo di ricostruzione è:

Dataset	Errore massimo di ricostruzione
venice_boat	$3.331 \cdot 10^{-16}$
living_room_window	$1.665 \cdot 10^{-16}$
venice_carnival	$2.220 \cdot 10^{-16}$

10.3 Coerenza delle shape

Il terzo controllo verifica che ogni livello Gaussiano dei pesi abbia la stessa altezza e larghezza del corrispondente livello Laplaciano delle immagini. Questa condizione è necessaria per calcolare il prodotto $G_l(\hat{W}_k) \cdot L_l(I_k)$ senza ambiguità dimensionali. Sui dataset verificati, la coerenza delle shape risulta soddisfatta per tutti i livelli.

11 Differenze implementative e limiti

L'implementazione segue la struttura principale del metodo, ma include alcune scelte pratiche:

- le immagini sono rappresentate come array NumPy RGB in $[0, 1]$;
- i JPEG sono trattati come immagini display-referred, senza linearizzazione sRGB, gamma inversa o gestione dei profili colore;
- il contrasto usa un kernel Laplaciano discreto con gestione dei bordi `reflect`;
- la saturazione cromatica è stimata come deviazione standard dei canali RGB;
- la well-exposedness usa una Gaussiana centrata a 0.5 con `sigma=0.2`;
- le piramidi sono costruite tramite funzioni di `scikit-image`, quindi downsampling, upsampling e gestione dei bordi non sono garantiti bit-exact rispetto ai filtri piramidali originali Burt-Adelson usati come riferimento teorico;
- i pesi vengono rinormalizzati anche ai livelli piramidali, come scelta implementativa per stabilità numerica;
- `fuse_exposures` clippa per default l'output finale in $[0, 1]$ dentro `fused_image`, mentre `save_rgb_image` applica un ulteriore clipping prima del salvataggio;
- il progetto è orientato allo studio dell'algoritmo, non alla riproduzione bit-exact del codice originale.

Queste scelte non cambiano il nucleo della pipeline, ma vanno dichiarate per evitare claim troppo forti sulla corrispondenza con il paper.

Il metodo assume inoltre che le immagini della sequenza siano allineate. Se la camera o i soggetti si muovono tra gli scatti, la fusione può produrre ghosting o artefatti locali. L'implementazione presentata non include registrazione delle immagini o compensazione del movimento. Anche rumore, clipping estremo, bilanciamento colore incoerente e scelta degli esponenti ω possono influenzare il risultato.

Infine, Exposure Fusion produce direttamente una immagine LDR. Questo è utile quando si vuole evitare la pipeline HDR completa, ma significa che il risultato non rappresenta una radiance map fisicamente calibrata della scena.

11.1 Elementi non implementati rispetto al paper

Il paper discute anche estensioni e casi applicativi che non sono stati implementati in questo progetto. In particolare, la pipeline non include:

- registrazione automatica o compensazione del movimento tra immagini;
- gestione dedicata di sequenze flash/no-flash;
- tuning interattivo degli esponenti ω ;
- una valutazione percettiva sistematica su un numero ampio di dataset;
- confronto quantitativo con operatori HDR o tone mapping alternativi.

Queste omissioni delimitano il perimetro del progetto: l'obiettivo è riprodurre e studiare la pipeline principale di Exposure Fusion, non implementare tutte le varianti discusse dagli autori.

12 Conclusioni

Il progetto implementa in Python il nucleo dell'algoritmo Exposure Fusion:

`quality-based pixel weighting + pyramid-based multiresolution blending`

La corrispondenza con il paper è visibile nella struttura matematica e nella suddivisione del codice: `weights.py` calcola le misure di qualità e le mappe di peso, `pyramids.py` gestisce le decomposizioni multirisoluzione e `fusion.py` combina immagini e pesi livello per livello. I controlli numerici mostrano che i pesi sono normalizzati, che le piramidi Laplaciane sono ricostruibili entro la precisione floating point e che i livelli piramidali hanno shape compatibili.

Il risultato è una pipeline semplice, ispezionabile e coerente con il metodo di Mertens, Kautz e Van Reeth. L'analisi del parametro `max_layer` mostra inoltre che la profondità della piramide ha un effetto misurabile su MAE e MSE rispetto alle reference disponibili e che conviene trattarla come scelta sperimentale legata al dataset, non come costante universale né come parte prescritta dal paper originale. Possibili sviluppi futuri includono test automatici, analisi quantitativa su più dataset, metriche complementari come SSIM e PSNR senza trattarle come giudizi definitivi, registrazione delle immagini per sequenze non perfettamente allineate e studio sistematico dell'effetto degli esponenti ω sulla qualità finale.

13 Riferimenti

- T. Mertens, J. Kautz, F. Van Reeth, *Exposure Fusion*, Proceedings of the 15th Pacific Conference on Computer Graphics and Applications (Pacific Graphics), 2007.
- `exposure-fusion.pdf`, copia del paper di riferimento inclusa nel repository.
- `exposure_fusion_study.ipynb`, notebook di studio e implementazione guidata dell'algoritmo.
- `exposure_fusion_core/`, package Python riusabile dell'implementazione.